LAMBA: REDUCING DOOM LOOPS AND ANSWER-LETTER BIAS WITHOUT LOSING CAPABILITY

TECHNICAL REPORT OF LAMBA 0.1 AND 0.2

Vixero Labs

Snapshot of 2 October 2026

ABSTRACT

We describe Lamba 0.1 and Lamba 0.2, two LoRA adapters on a 28.8B-parameter hybrid model from the Qwen3.5 family (48 Gated DeltaNet and 16 full-attention layers), built by one researcher on a single GPU. The base model had two defects: it falls into runaway repetition ("doom loops") on long reasoning, and on the Malay benchmark MalayMMLU it is 11.2 points less accurate when the answer is A than when it is B. Lamba 0.1 adapts Liquid AI's AntiDoom recipe to this architecture, mines loops from its own outputs and per language, and adds a separate letter-bias stage. On our 150-prompt loop test, runaway loops at the end of answers fall from 18.0% to 0%, and any repetition in the final 2,500 characters from 37.3% to 2.7%. A factorial comparison shows that training alone and a repetition penalty of 1.1 alone each remove most loops, by similar amounts; combined they leave the fewest loops, but on our pre-registered primary endpoint (exact repetition or token-budget exhaustion) all three give a similar improvement of about 25% relative on 700 prompts, short of the 30% we targeted. The A–B gap shrinks from -11.2 to -1.9 points on the full 23,903-item benchmark, although the checkpoint was chosen on that benchmark. GPQA-Diamond and MalayMMLU accuracy show no significant change. Lamba 0.2 adds a DPO stage on tool-use transcripts and shows no significant difference from Lamba 0.1 on the metrics we tested. We compare against eight other models on the same test sets and report the full lineage, the negative results, and the corrections we made to our own measurements.

Contents

1 Introduction **3**

2 Related work **5**

3 Setup **5**

4 Method **6**

5 Evaluation protocol **7**

6 Loop reduction **9**

7 Answer-letter bias **11**

8 Lamba 0.1 **12**

9 Tool use and Lamba 0.2 **12**

10 Comparison with other models **15**

11 Analysis: what the training changes **16**

12 Negative results **18**

13 Findings from live use **18**

14 Lessons for practitioners **19**

15 Limitations **19**

16 How this work was done **20**

17 Conclusion **20**

A Corrections **22**

B Training details **23**

C Compute **23**

D Reproducibility checklist **23**

E Prompts and harness **24**

F Reproduction **25**

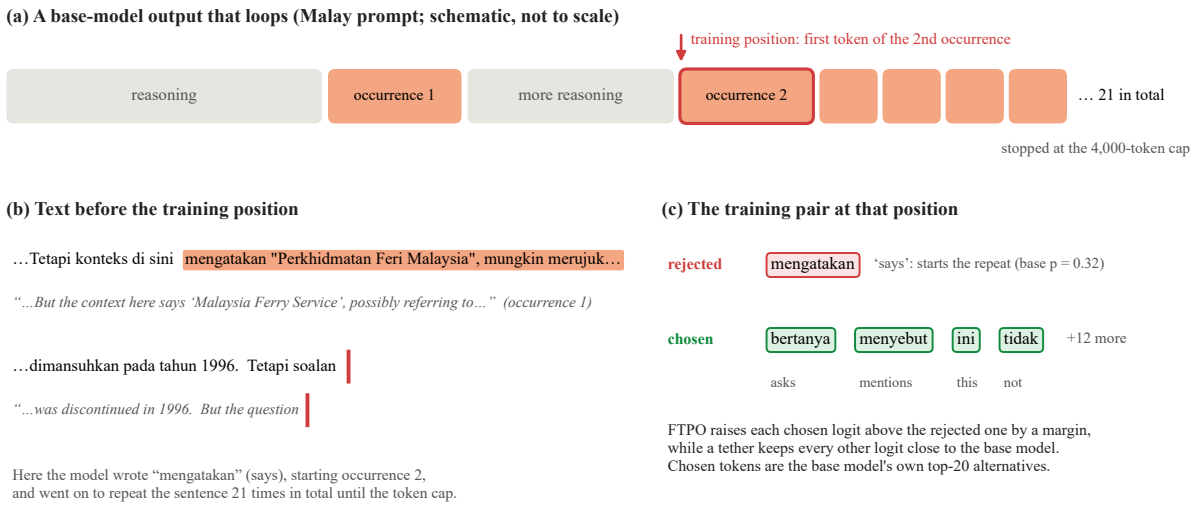


Figure 1: How an AntiDoom training pair is built, shown on a real pair from our training data. (a) The base model repeats a sentence 21 times and runs out of tokens. (b) The context is cut at the first token of the second occurrence. (c) The token the model actually produced there is rejected; the base model's own top alternatives are chosen. Glosses are English translations of the Malay.

1 Introduction

Reasoning models fail in a characteristic way: they start repeating a sentence, a list item or a line of arithmetic and do not stop until the token budget runs out. Liquid AI called these *doom loops* and showed that a preference-optimisation method applied at the token where a repeat begins (AntiDoom, built on final-token preference optimisation, FTPO) removes most of them in small models [10]. Our base model, a further-trained checkpoint of the Qwen3.5 architecture, loops often: on our held-out loop test, 39.3% of greedy answers contain a block of at least 60 characters repeated four or more times, and 18.0% end in a runaway loop.

The same model carries a positional prior on MalayMMLU [12]: when the correct answer is A it is right 71.6% of the time, when it is B, 82.9%. Moving the gold answer between positions shows this is a preference for answer positions, not a content effect.

Lamba is the work of one researcher at Vixero Labs, on one NVIDIA L40S. The first training run was on 19 September 2026; Lamba 0.1 shipped on 23 September and Lamba 0.2 on 2 October. Figure 1 shows how a training pair is built from a looping output, and Figure 2 how each training stage changed loop behaviour.

Contributions.

- **AntiDoom on a hybrid model.** We port AntiDoom to a 28.8B Gated DeltaNet hybrid, with an architecture-aware LoRA layout, mining on the adapter's own outputs, and a per-language round built from model-written prompts for a domain where natural loops are too rare to mine (§6).
- **Training versus decoding.** A 2×2 comparison (base or adapter, with or without a repetition penalty) on 700 prompts shows that training alone and the penalty alone are about equally effective, and that combining them removes the most loops (R_{any}: 225 base, 47 training only, 37 penalty only, 14 both).

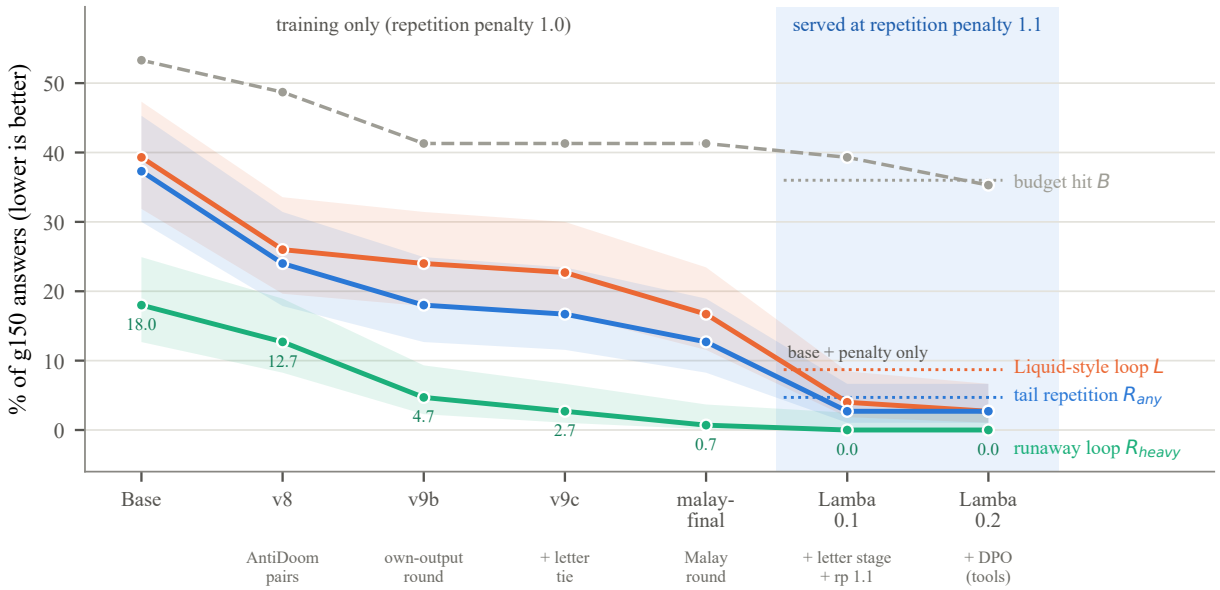


Figure 2: Loop rates on the 150-prompt loop test (g150) across the Lamba lineage, recomputed from raw outputs; shaded bands are 95% Wilson intervals. The first five points are evaluated without a repetition penalty, so they isolate the effect of training: runaway loops (R_{heavy}) fall from 18.0% to 0.7%. The shaded points are the shipped versions, served at repetition penalty 1.1; dotted lines mark the untouched base model with the same penalty, which achieves most of the same reduction on its own (§6.5). Budget hits (dashed) fall less because many `mcp` prompts need long but legitimate solutions (Table 4).

A penalty of 1.2 makes loops worse than the base model (§6.5).

- **Letter bias and checkpoint instability.** A group objective over cyclic option rotations reduces the A–B gap from -11.2 to -1.9 points, and we document that letter-bias checkpoints of a single run range from -13.7 to $+11.7$ points on the full benchmark (§7).
- **Measurement.** A frozen metric for redundant tool calls after a verified answer; the finding that most of Lamba 0.1's tool failures came from the evaluation sandbox, not the model; and a pre-registered loop endpoint reported under both the planned and a substituted detector (§5, §9).
- **An open record.** Results for eight other models on the same harness (§10), every negative result (§12), the corrections we made along the way (Appendix A), and the code for the trainer, the loop and tool evaluations and the scorers.

Key findings.

1. *Training and the penalty are interchangeable, and complementary.* On 700 prompts, training alone cuts runaway loops from 16.6% to 1.0% and the penalty alone to 0.3%; together they leave 2.0% tail repetition against 6.7% and 5.3% for each alone. On budget hits and the primary endpoint the three configurations are indistinguishable (§6.5).
2. *Learning from re-quoting loops generalises.* 80% of the first-round training pairs come from loops in which the model re-quotes the question, yet our evaluation, which ignores text copied from the prompt, shows loops in the model's own words falling sharply, including on science and Malay prompts with little overlap with training data (§11).

3. *The training position is everything.* One position earlier than the start of the repeat, the same pipeline taught the model to stop writing newlines and made loops worse (§11).
4. *The useful penalty window is narrow.* At 1.2 the penalty pushes 87% of answers into the token cap, against 41% without it (§11).
5. *Results replicate across seeds.* Retraining the last loop stage with a new seed changes no loop or capability metric significantly (§11).
6. *Bias checkpoints are unstable.* Seven checkpoints of one letter-bias run range from -13.7 to $+11.7$ points (§7).

When given its system prompt, Lamba introduces itself as "Lamba, built by Vixero Labs". It is built on a Qwen-family base, and its model card says so.

2 Related work

Repetition in neural text generation. Greedy and beam decoding produce repetitive text even from strong language models [6]. Training-time remedies penalise repeated tokens directly, as in unlikelihood training [15], or teach the model to escape sentence-level loops using synthetic repetitive data, as in DITTO [16], which also showed that repetitions reinforce themselves. Decode-time remedies include the repetition penalty introduced with CTRL [8], which we use, and nucleus sampling. AntiDoom [10] targets the long loops of reasoning models with a single-token preference objective at the start of the first repeat. We follow it closely, scale it to a 28.8B hybrid model, and measure how much of the gain comes from training and how much from the penalty.

Answer-position bias. Multiple-choice accuracy of language models changes when options are reordered [11], largely because of a prior over option identifiers [18]. Proposed fixes include inference-time debiasing [18], fine-tuning on permutation-consistency objectives with LoRA [5] and reinforcement learning that rewards consistent answers across permutations [19]. Our letter stage is a supervised group objective in the same family, applied to a Malay benchmark.

Efficient reasoning. Reasoning models often spend many tokens on problems that need few [3]. ThinkingCap [1] fine-tunes the same base architecture as Lamba for shorter reasoning, and we compare against it directly. Our own efficiency experiments were negative or inconclusive (§12), and we leave verbosity to future work.

3 Setup

Base model. The base (`soup-linear-l1.0`) is a 28.8B-parameter model with the architecture of Qwen3.8-27B: 64 layers, hidden size 5120, 48 Gated DeltaNet layers [17] interleaved with 16 full-attention layers. We continued training it before this work and treat it as fixed here.

Hardware. All training and evaluation ran on a single NVIDIA L40S (48 GB, about 46 GB usable) with 30 GB of host RAM; a second identical machine was used on a few days for evaluation. The base cannot be held in bf16 on the card, so training uses 4-bit (NF4) QLoRA [4] and serving uses online fp8 quantisation. Appendix C gives run times.

Software. vLLM 0.28 [9] for generation, with multi-token-prediction speculative decoding (3 tokens) and the Qwen3 reasoning and XML tool-call parsers; PyTorch 2.11 with CUDA 13.4; fused Gated DeltaNet kernels (`fla-core` 0.5.2, `causal-conv1d` 1.7.0), which cut training step time from 405 s to 131 s; and a trainer derived from Liquid AI's AntiDoom code with our own losses.

Adapter scheme. LoRA [7] with per-module ranks: 128 on attention q, k, v, o; 64 on MLP projections; 64 on the Gated DeltaNet input projection for q, k, v, 32 on its gate projection, 64 on its output projection; 16 on the LM head. Adapters cover all 16 full-attention layers and the last 24 Gated DeltaNet layers. vLLM cannot apply LoRA to the LM head, so we merge only the LM-head delta, with stochastic rounding, into a copy of that weight. Per-stage hyperparameters are in Appendix B.

Why the adapter is not merged. For our adapters the fp8 quantisation noise is three to eight times the size of the LoRA delta under every rounding scheme we tried, so merging would erase it. vLLM computes the base output and the LoRA correction separately, so serving the unmerged adapter on the fp8 base preserves the delta. Every evaluation in this report uses that path.

Decoding. Loop and capability evaluations are greedy. Lamba and its base run at reasoning effort "medium" in every evaluation; the GPQA prompts additionally carry the harness's "xhigh" instruction text, identically for the base and all adapters. Lamba is served with repetition penalty 1.1 unless stated otherwise. Single-stream serving runs at 14.8 tokens/s without speculative decoding and 41.6 tokens/s with three draft tokens.

4 Method

All training objectives act on the next-token logits at a single position. We write $\boldsymbol{z} \in \mathbb{R}^V$ for the adapter's logits, $\boldsymbol{z}^0$ for the logits of the frozen base model (the same network with the adapter disabled), and $p_\theta = \mathrm{softmax}(\boldsymbol{z})$, $p_0 = \mathrm{softmax}(\boldsymbol{z}^0)$.

Mining loop pairs. Algorithm 1 turns a looping output into a preference pair (Figure 1). The position is the first token of the second occurrence of the repeated unit; the rejected token r is what the model produced there, and the chosen set C is the base model's top-20 alternatives at the same context.

FTPO objective. For a pair (c, r, C) let $\Delta_t = z_t - z_r$ for each chosen token $t \in C$, and $w_t = \mathrm{clip}\big((\varepsilon - \Delta_t)/\varepsilon, 0, 1\big)$ with margin $\varepsilon = 1.5$. The preference loss pushes every chosen logit above the rejected one by the margin and stops pushing once it is reached:

$$\mathscr{L}_{\mathrm{pref}} = \frac{1}{|C|} \sum_{t \in C} w_t \, \mathrm{softplus}\big(\varepsilon - \Delta_t\big). \tag{1}$$

A tether keeps all other logits close to the base model and limits how far the target logits move:

$$\mathscr{L}_{\mathrm{tether}} = \lambda \operatorname*{mean}_{v \notin C \cup \{r\}} \big(z_v - z_v^0\big)^2 + \lambda_t \operatorname*{mean}_{v \in C \cup \{r\}} \max\big(|z_v - z_v^0| - \tau_t, 0\big)^2, \tag{2}$$

with $\lambda = 2.0$ for v8 and 0.1 afterwards, $\lambda_t = 0.1$ and $\tau_t = 0.1$. The loss for a hard pair is $\mathscr{L}_{\mathrm{pref}} + \mathscr{L}_{\mathrm{tether}}$.

Algorithm 1 Mining AntiDoom pairs from model outputs

Require: outputs $\mathscr{Y}$ with prompts; base model f_0; window w=60 characters; minimum count k=4
for each output y with prompt x **do**
  $\mathscr{W} \leftarrow$ windows of w characters, anchored every $w/2$, with $\geq$25 alphanumeric characters, occurring $\geq k$ times in y without overlap
  for each repeated window in $\mathscr{W}$ **do**
    $p_1 \leftarrow$ start of its second occurrence; $p_0 \leftarrow$ start of its first
    extend p_0, p_1 leftward while $y[p_0-1] = y[p_1-1]$, then snap p_1 to the next word start
    $c \leftarrow x \oplus y[:p_1]$ ▷ context ends just before the repeat
    $r \leftarrow$ first token of $y[p_1:]$ ▷ rejected: starts the repeat
    $C \leftarrow$ top-20 tokens of $f_0(c)$ other than r, excluding non-alphanumeric tokens
    if $\log p_0(r) < -0.5$ **and** $\max_{t \in C} \log p_0(t) - \log p_0(r) > -5$ **then**
      keep (c, r, C) ▷ skip near-certain repeats; require a plausible alternative
    end if
  end for
end for
keep at most two pairs per output; cap pairs per rejected token; remove duplicates

Soft rows. Later occurrences of a repeat, and contexts where the base model is nearly certain of the repeated token, use a soft target instead of a hard pair:

$$q = \tfrac{1}{2}\, p_0 + \tfrac{1}{2}\, \mathrm{softmax}\Big(\frac{\log p_0 - \gamma \boldsymbol{e}_r}{\tau}\Big), \qquad \mathscr{L}_{\mathrm{soft}} = \omega\, \mathrm{KL}\big(q \,\|\, p_\theta\big), \tag{3}$$

where $\boldsymbol{e}_r$ is the indicator of the rejected token, $\gamma = 4$, $\tau = 0.7$, and $\omega \leq 1$ is a per-row weight. With $\gamma = 0$ and $\tau = 1$ the target is $q = p_0$; we use such rows to tie the adapter to the base model on non-looping text and on answer letters.

Letter-group objective. A training example is a group of K renderings of one question, each a cyclic rotation π_k of the options. In rendering k the chosen token is the gold letter g_k and the rejected token $\bar{g}_k$ is the base model's highest-scoring wrong letter. With $\boldsymbol{a}_k$ the probabilities over the option letters and $\tilde{a}_{k,j}$ the probability given to original option j (undoing π_k),

$$\mathscr{L}_{\mathrm{letter}} = \alpha \frac{1}{K} \sum_k w_k\, \mathrm{softplus}\big(\varepsilon - (z_{k,g_k} - z_{k,\bar{g}_k})\big) + \lambda_{\mathrm{pv}} \frac{1}{n} \sum_j \operatorname*{var}_k \big(\log \tilde{a}_{k,j}\big) + \lambda_H \frac{1}{K} \sum_k H(\boldsymbol{a}_k) + \lambda\, \mathscr{L}_{\mathrm{tether}}^{\neg\mathrm{letters}}, \tag{4}$$

where the second term penalises the same option receiving different probability under different rotations, the entropy term discourages flat letter distributions, and the tether excludes all option letters. Lamba 0.1 uses $\alpha = 0.4$, $\lambda_{\mathrm{pv}} = 0.2$, $\lambda_H = 0.05$.

DPO stage. Lamba 0.2 adds the standard DPO loss [13] on full responses, combined with the terms listed in §9.

5 Evaluation protocol

5.1 Loop test

The main loop test (g150) is 150 loop-prone prompts that were never used to build training pairs: 65 from the `mcp` domain, 65 science and 20 Malay[1]. Many `mcp` prompts share templated wording with training prompts

[1] `mcp`: competition mathematics and competitive programming problems from our internal corpus.

(60 of 65 have at least 50% 8-gram overlap with some training prompt); science and Malay prompts have little overlap, and the loop reduction holds on them separately (Table 4). Generation is greedy with a 4,000-token cap that includes the reasoning. For each output we compute:

- L (Liquid-style loop): some window of $\geq$60 characters (with $\geq$25 alphanumeric) occurs $\geq$4 times anywhere in the output.
- R_{any}: any such repeated text inside the last 2,500 characters.
- R_{heavy}: more than 1,500 repeated characters inside the last 2,500 characters, i.e. a runaway loop at the end.
- B (budget hit): generation stopped at the 4,000-token cap.

Windows that also occur in the prompt are excluded, so quoting the question is not counted. Comparisons are paired by prompt and tested with an exact McNemar test; confidence intervals on relative reductions use 2,000 bootstrap resamples over prompts. These detectors are from the same family as the one used to mine training pairs, so we also ran two independent checks on earlier adapters in the lineage: a zlib compression-ratio test with no repetition detector (on v8) and blind adjudication of 40 output pairs by a separate model (on v9c: 10 base outputs and 1 adapter output judged to end in a runaway loop, $p = 0.004$). Both agreed in direction.

Pre-registered endpoint. Before any training we fixed the success rule: on held-out loop-prone prompts, the rate of *exact repetition or budget hit* must fall with one-sided McNemar $p < 0.025$, by at least 30% relative and at least 10 points absolute. "Exact repetition" is defined by the inner-repetition detector shipped with the AntiDoom code. The original rule also counted a semantic near-duplicate detector; after seeing base-model data, and before seeing any adapter data, we removed it because it fired on 69 of 70 base outputs. Later in the project, internal analyses computed the same endpoint with the Liquid-style detector L in place of the inner-repetition detector. That substitution was not a planned change, and we report both versions (Table 2).

5.2 Other evaluations

- **GPQA-Diamond** [14]: 198 questions from an ungated mirror containing the same problems, greedy, 4,000-token budget with a forced answer at the cap. Between 51% and 59% of answers are cut at the cap depending on the model, so absolute accuracy is a proxy and we only compare at equal budgets.
- **MalayMMLU** [12]: all 23,903 items whose gold answer is A–D, scored by the log-probability of the answer-letter token. We report accuracy, position-balanced accuracy (mean of per-gold-letter accuracy), the A–B accuracy gap, the standard deviation of per-position recall (RStd) and a calibration divergence (CKLD) following Zheng et al. [18].
- **Shift test**: 5,000 MalayMMLU items under every cyclic rotation of the options; consistency is the share of questions answered with the same content under all rotations, and shift-RStd is the spread of accuracy across positions.
- **eval30**: 30 maths tasks with sympy-verified answers (12 definite integrals and 18 across ODEs, series, linear systems, limits, combinatorics and matrices), 8 runs each at temperature 0.8, with a Python tool in a sandbox and at most 8 tool rounds, scored with a frozen metric (§9).

We report many paired tests without correction for multiple comparisons; isolated results with $0.01 < p < 0.05$ should be read as weak evidence.

6 Loop reduction

6.1 What did not work first

Our first attempt mined preference pairs from existing base-model rollouts and trained FTPO on them. It had no effect, and later versions made loops worse. Two bugs explain this. The miner used the first occurrence of a repeated span as the training position rather than the start of the second occurrence, so the "rejected" tokens were ordinary text: 15% were the paragraph break `\n\n` and many others were sentence openers such as "Then" and "The". Training suppressed newlines (from 7.11 to 0.03 per 1,000 characters) and produced run-on text that looped more. In addition, 89% of the pairs came from a semantic-similarity detector that fired on 76% of all rollouts, so most of them were not loops.

6.2 AntiDoom, as specified

The version that worked follows Liquid AI's recipe [10]. A loop is a 60-character window repeated at least four times. The training position is the first token of the *second* occurrence of the repeated unit (the first repeat), after extending the match leftward to its true start and snapping to a word boundary. The rejected token is the one the model produced there; chosen tokens are the base model's top-20 alternatives, excluding non-alphanumeric tokens.

From 81,661 existing rollouts this gave 22,023 loop events. We kept pairs where the base model was not already near-certain of the rejected token (log-probability below -0.5) and the best alternative was within 5 nats, leaving 1,298 pairs. This adapter (v8) was the first with a clear effect: on 586 prompts not used for training, the Liquid-style loop rate fell from 52% to 32% ($p = 7 \times 10^{-16}$), while the worst runaway loops at the end of outputs changed little (Figure 2).

6.3 Rounds on the adapter's own outputs

Round 2 mined new loops from v8's outputs on prompts disjoint from all evaluation sets. The second occurrence of each loop became a hard FTPO pair (Eqs. 1–2); the third and fourth became soft rows (Eq. 3). 909 mined rows plus 120 rows tying the adapter to the base on non-looping text trained v9b, starting from v8. This is the round that removed most runaway loops (R_{heavy} 12.7% $\rightarrow$ 4.7%). v9c added 1,080 rows tying answer-letter probabilities to the base on multiple-choice prompts, because v9b had shifted the letter prior (MalayMMLU fell by 1 point). The tie cost nothing on loops and restored MalayMMLU.

6.4 Malay round

v9c reduced loops on the domains it was mined from, but on 180 unseen Malay multiple-choice prompts the reduction was small and not significant. Real Malay prompts loop rarely (3.4%), so mining them gave too few pairs. We had a separate model write Malay prompts likely to elicit loops, ran our own model on them (about 15% looped) and mined pairs from those outputs. The mix was 273 Malay rows, 600 replay rows from earlier rounds and 200 letter-tie rows, trained for one epoch from v9c. On 400 held-out Malay prompts this adapter (malay-final) reduced R_{any} from 7.8% to 2.5% and R_{heavy} from 5.2% to 1.5% ($p < 0.001$). On the 180 of those prompts with gradable answers, accuracy rose from 90.6% to 94.4%, mostly because fewer answers were cut off.

6.5 Training versus decoding

To separate what training contributes from what the decoding setting contributes, we ran both the base model and malay-final (the last loop-training stage before the letter stage) with and without repetition penalty 1.1, on g150 and on the 700-prompt set (Table 1).

Table 1: Training versus decoding: base model and malay-final, each with and without repetition penalty 1.1 (counts). Paired exact McNemar tests in the text.

Test set	Configuration	L	R_{any}	R_{heavy}	B
g150 (150)	Base, no penalty	59	56	27	80
	Base + penalty only	13	7	0	54
	Training only (malay-final)	25	19	1	62
	Training + penalty	6	2	0	55
700 prompts	Base, no penalty	248	225	116	364
	Base + penalty only	52	37	2	271
	Training only (malay-final)	69	47	7	274
	Training + penalty	21	14	1	280

- **Each alone removes most loops.** On 700 prompts, training alone takes runaway loops from 116 to 7 and tail repetition from 225 to 47; the penalty alone takes them to 2 and 37. The two are comparable, with the penalty slightly ahead on the loop metrics.
- **Combined is best on loops.** Adding training on top of the penalty roughly halves the loops that remain: Liquid-style loops 52 $\rightarrow$ 21 ($p = 9 \times 10^{-5}$) and tail repetition 37 $\rightarrow$ 14 ($p = 4 \times 10^{-4}$).
- **Budget hits do not care which.** Training only, penalty only and both give 274, 271 and 280 budget hits out of 700 (base: 364). Most remaining budget hits are long but legitimate solutions (Table 4), which neither lever addresses.

The practical upshot is that training makes the model loop-resistant without depending on a decoding setting that users may not apply, and the penalty, which costs nothing, adds a further reduction when it is applied. The capability cost of the penalty on the base model is untested; with the adapters, GPQA-D and MalayMMLU are unchanged (Table 3).

A sweep of the penalty on top of malay-final (Figure 3) shows a narrow useful window: 1.1 helps, 1.15 helps less than no penalty with the Liquid-style detector and barely at all with the pre-registered one, and 1.2 produces more loops than the base model. Outputs at 1.1 remain coherent and well formatted; newline frequency drops by about 30%. The penalty was chosen on g150, so g150 results at 1.1 are not an independent test of it.

6.6 The primary endpoint

Table 2 gives the primary endpoint, against the base model without a penalty, under both detectors. With the pre-registered detector every configuration improves significantly, but on the 700-prompt set (never used for training or for choosing the penalty, although earlier adapters had been evaluated on it) training only, penalty only and both together all land between 22.8% and 25.3% relative, below the 30% target. The point estimates above 30% occur only on g150, where the penalty was selected. On top of the penalty, the adapters do not change the endpoint at all (relative change -3.3% on 700 prompts, CI -11.6 to 3.9). The endpoint is dominated by budget hits, many of which are long but legitimate solutions (Table 4), which is why it moves much less than the loop metrics.

Summary of the loop result. Training alone and a repetition penalty alone each remove most loops, by similar amounts; combined they leave the fewest (R_{any} 2.0% and R_{heavy} 0.1% on 700 prompts, from 32.1% and 16.6%). The pre-registered primary endpoint improves by about 25% relative under any of the three configurations, short of the 30% target, because it is dominated by long but legitimate answers that hit the token cap.

Table 2: Primary endpoint (exact repetition or budget hit), each configuration paired against the base model without a penalty. "Pre-reg." uses the pre-registered inner-repetition detector; "Liquid" substitutes the Liquid-style detector L. Relative reductions with 95% bootstrap CIs.

Configuration	Test set	Pre-reg. rate	Rel., pre-reg.	Rel., Liquid
Penalty only (base, rp 1.1)	g150	53.3 → 36.0	32.5% (18.8–45.3)	34.1% (22.2–45.8)
Training only (malay-final)	g150	53.3 → 41.3	22.5% (8.7–35.8)	23.1% (10.9–35.1)
Lamba 0.1 (rp 1.1)	g150	53.3 → 39.3	26.3% (12.5–38.6)	33.0% (19.8–44.9)
Lamba 0.2 (rp 1.1)	g150	53.3 → 35.3	33.8% (18.8–47.6)	39.6% (26.0–52.1)
Penalty only (base, rp 1.1)	700 prompts	52.0 → 38.9	25.3% (19.5–31.2)	29.5% (24.1–35.3)
Training only (malay-final)	700 prompts	52.0 → 39.3	24.5% (18.7–30.1)	28.5% (23.5–33.9)
Training + penalty (rp 1.1)	700 prompts	52.0 → 40.1	22.8% (16.6–28.5)	31.2% (25.7–37.0)

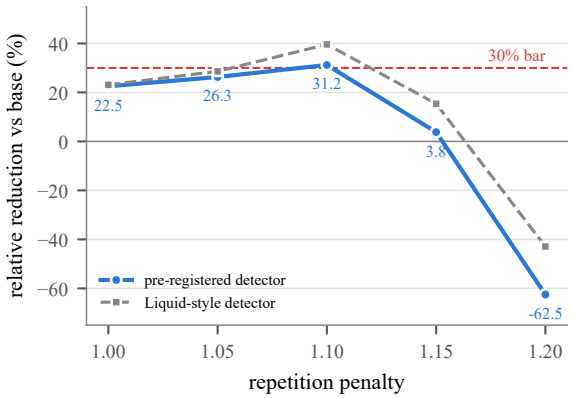


Figure 3: Primary-endpoint reduction versus base for malay-final on g150, by repetition penalty, under both detectors. The penalty was selected from this sweep.

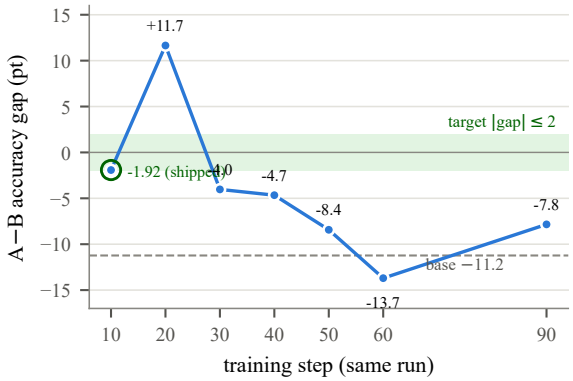


Figure 4: A–B gap on the full MalayMMLU for seven checkpoints of one letter-bias run.

7 Answer-letter bias

Diagnosis. On the base model, accuracy by gold letter is 71.6 / 82.9 / 75.5 / 81.3 for A–D. Rotating the options moves the error with the position, and placing options inline instead of on separate lines reverses the pattern, so the bias is a position prior tied to the prompt format, consistent with Zheng et al. [18]. Averaging over permutations at inference gives 79.9%, an upper bound on what removing position effects alone could recover.

Data. We used multiple-choice questions from our own corpus, never benchmark items: questions whose gold answer could be verified, after removing exact and near duplicates of MalayMMLU and GPQA items. Each question is rendered under every cyclic rotation of its options. We kept questions the base model answers inconsistently across rotations, plus 25% consistent ones as anchors, and held out 15% for validation.

Objective. Each training example is a group of rotations of one question, trained with Eq. 4: a hinge on the correct letter, a penalty on the variance of each option's probability across rotations (related to Guda et al. [5] and Zheng et al. [19]), an entropy term that prevents collapse to a uniform prediction, and a tie to the base model on all non-letter tokens.

Table 3: Base, Lamba 0.1 and Lamba 0.2. Loop metrics on g150 (%, lower is better). Both adapters at repetition penalty 1.1; the last rows show the base model with the penalty alone and Lamba 0.2 without it.

	L	R_{any}	R_{heavy}	B	Tokens‡	GPQA-D	MalayMMLU	A–B gap
Base	39.3	37.3	18.0	53.3	2969	151/198	76.74	−11.2
Lamba 0.1	4.0	2.7	0.0	39.3	2765	147, 153†	77.71	−1.9
Lamba 0.2	2.7	2.7	0.0	35.3	2694	157/198	77.91	+1.7
Base, rp 1.1	8.7	4.7	0.0	36.0	2706	–	–	–
Lamba 0.2, rp 1.0	14.0	10.0	2.0	35.3	2646	–	–	–

† Two runs with identical settings; greedy decoding in vLLM is not bit-reproducible across batch compositions. Neither differs significantly from base. ‡ Mean tokens are capped at 4,000 and therefore understate the length of truncated answers.

Overshoot and checkpoint instability. The first letter adapter over-corrected, from −11.2 to +14.5. The final run starts from malay-final with the letter losses scaled down (hinge 0.4, permutation variance 0.2, entropy 0.05). Its in-training validation set (664 questions) showed a gap between 2 and 6 points at every evaluation, but on the full benchmark the run's checkpoints range from −13.7 to +11.7 (Figure 4), with a 15.7-point swing between steps 20 and 30.

How the checkpoint was chosen. We first scored steps 30, 60 and 90 on the full benchmark and initially shipped step 30 (gap −4.0). We then scored steps 10, 20, 40 and 50 on the same benchmark; step 10 was the only one within the ≤2-point target and became Lamba 0.1. The benchmark was therefore used for checkpoint selection, and given the size of the swings the reported gap is likely optimistic. A held-out split of MalayMMLU or a second Malay benchmark would be needed for an unbiased estimate.

What changed. Lamba 0.1 reduces the gap from −11.2 to −1.9 and RStd from 4.5 to 1.98. Position-balanced accuracy moves only from 77.8% to 78.1%, and permutation-averaged accuracy on the shift test is unchanged (79.9% vs 79.8%). Most of the 1-point gain in raw accuracy comes from predicting A more often (37.6% → 41.3% of predictions) on a benchmark where 45% of gold answers are A. On the shift test, consistency rises from 74.8% to 80.1% and shift-RStd falls from 8.8 to 6.5, but accuracy still differs by position (73.1% in the first position versus 83.1% in the second). The bias is reduced, not removed.

8 Lamba 0.1

Lamba 0.1 is the step-10 checkpoint of the letter run, served with repetition penalty 1.1. Table 3 compares it to the base model and to Lamba 0.2, Table 4 breaks the loop results down by domain, and Figure 5 shows a typical case. Under sampled decoding (temperature 0.7, 100 prompts), its predecessor malay-final reduced R_{any} from 29% to 0%.

9 Tool use and Lamba 0.2

9.1 Measuring redundant tool calls

Every eval30 prompt asks the model to compute an exact value and then verify it, so the first verification call is required. Our first metric ignored that and reported that 37.5% of runs "ruminated"; it also counted crashed checks, legitimate repairs and follow-ups after a mismatch. The frozen metric (v2.2) defines a *done point*: the exact value has appeared in a tool output and a successful call using the task family's independent verification method has completed. Calls after the done point are redundant. The metric was checked by

Table 4: g150 results by domain (counts). Loops fall in every domain, including science and Malay, whose prompts share little wording with training data. `mcp` budget hits do not change: these are mostly long competition solutions rather than loops.

Domain (n)	Model	L	R_{any}	R_{heavy}	B
`mcp` (65)	Base	16	14	3	41
	Lamba 0.1	1	1	0	42
	Lamba 0.2	2	2	0	40
Science (65)	Base	33	32	16	30
	Lamba 0.1	4	3	0	16
	Lamba 0.2	2	2	0	12
Malay (20)	Base	10	10	8	9
	Lamba 0.1	1	0	0	1
	Lamba 0.2	0	0	0	1

Prompt (Malay physics, g150): *A metal is illuminated with ultraviolet light; its threshold frequency is* 6.0×10^{14} *Hz. If its work function is* 4.0×10^{-19} *J, calculate the maximum kinetic energy of the emitted photoelectrons (*$h = 6.63 \times 10^{-34}$ *J s).* The question omits the light's frequency.

Base model (stopped at the 4,000-token cap, last lines of its reasoning):

```
...the UV light has a frequency of 6.0 x 10^14 Hz.  In this case, KE = 0.  But that seems
too trivial.  Actually, I think I've been overthinking this.  Let me just go with the most
straightforward interpretation:  The metal has a threshold frequency of 6.0 x ...
```

Across the output, "I think I've been overthinking this" appears 5 times and "Let me just go with the most straightforward interpretation" 4 times; no answer is given.

Lamba 0.1 (1,486 tokens, finished): checks that the data are consistent ($\Phi = hf_0 = 3.978 \times 10^{-19} \approx 4.0 \times 10^{-19}$ J), notes that no other frequency is given so $f = f_0$, applies $K_{\max} = hf - \Phi$, and answers $K_{\max} = 0$ J, explaining in Malay that at the threshold frequency the photon energy only just overcomes the work function.

Figure 5: A g150 prompt on which the base model loops and Lamba 0.1 finishes. The base excerpt is verbatim from the evaluation output with line breaks removed and $\times$ written as x; Lamba's Malay answer is summarised in English.

hand on 13 labelled transcripts (all 5 flagged cases were correct; 2 known cases were missed) and parses fractions, LaTeX and Unicode minus signs. With it, redundant calls after verification are rare for most models (3–10%).

9.2 The sandbox was the main cause of tool failures

In our original sandbox `__import__` was blocked, and Lamba 0.1 never reached the correct answer in 37.5% of runs on the 12 integral tasks. Every one of those runs involved import errors: the model expects `import` to work and spent its tool budget rediscovering that it does not. A sandbox that allows a whitelist of maths libraries (sandbox-B) rescued 29 of 36 such runs (80.6%) within the original 8-round budget, with a median of 3 rounds. Stock Qwen3.8-27B failed at a similar rate (30.2%) in the original sandbox, which points to the harness rather than the model. All eval30 numbers in this report use sandbox-B.

While building it we found that `sympify()` could execute arbitrary Python in both sandboxes. Tool code now runs inside a bubblewrap jail [2] with no network, a private filesystem and an empty environment, with the Python-level restrictions kept as a second layer.

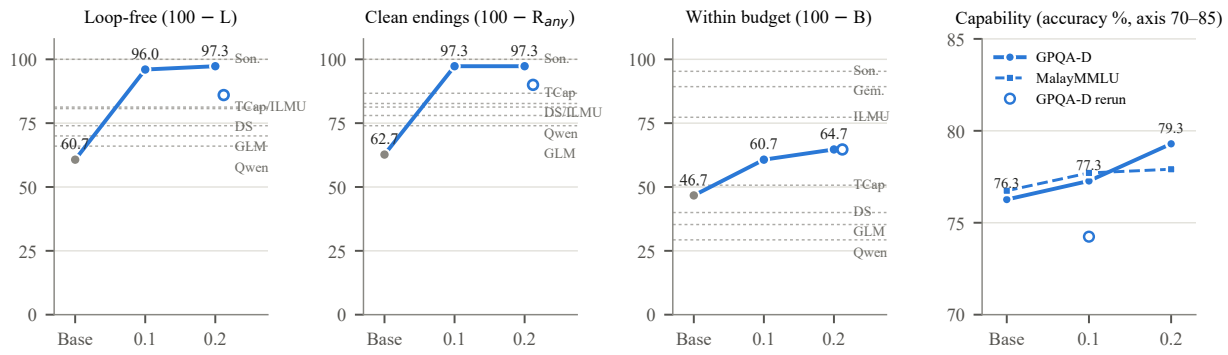


Figure 6: Base → Lamba 0.1 → Lamba 0.2 on g150 and on capability benchmarks; higher is better on every panel. Dashed lines are other models on the same test, for reference. The hollow marker in the first three panels is Lamba 0.2 at repetition penalty 1.0; in the last panel it is Lamba 0.1's second GPQA run. The capability panel uses a 70–85 axis.

9.3 DPO stage

Lamba 0.2 is a DPO [13] stage on 406 preference pairs built from tool transcripts in which a runtime intervention had led the model to a correct answer. It is initialised from, and uses as its DPO reference, an earlier tool-use adapter trained on top of Lamba 0.1 (seed 1337, §9.4). The loss is the DPO loss ($\beta = 0.1$) plus an SFT term on chosen responses (weight 0.05), a KL term to a frozen copy of Lamba 0.1 on replay data (weight 0.10), a smaller KL term to Lamba 0.1 on the SFT rows (0.01), and a logit-matching regulariser (0.1). Training ran for 4 epochs (148 steps) at learning rate 5×10^{-6}. The pairs had a formatting error: the chosen side was only the final answer, without the reasoning block or end token, and the rejected side was flattened prose.

At repetition penalty 1.1 we found no significant difference between Lamba 0.2 and Lamba 0.1 on any eval30 metric, on g150, on GPQA-D (157 vs 153 and 147 correct; paired $p = 0.54$ and 0.08) or on MalayMMLU (77.91% vs 77.71%; $p = 0.07$); see Tables 3 and 6. We did not pre-specify non-inferiority margins, so we describe this as "no significant difference on the metrics tested". On the integral-reconciliation family described next, where its initialisation adapter performed poorly in the original sandbox, Lamba 0.2 and Lamba 0.1 are both near zero in sandbox-B (Table 5). At repetition penalty 1.0, Lamba 0.2 loops more (R_{any} 10.0% vs 2.7% at 1.1, paired $p = 0.007$), so it is served at 1.1.

9.4 An unshipped track: post-correct rumination

Two further adapters (seeds 1337 and 42) were trained to reduce tool-call rumination. Both passed our loop, capability and letter checks, but on a family of seven integral-reconciliation tasks, evaluated in the original sandbox that blocked imports, they more often reached the correct value and then kept calling tools until the budget ran out without answering (Figure 7). In that sandbox Lamba 0.1 did so in 16% of runs and stock Qwen3.8-27B, a related untrained model, in 32.9%. On eight non-integral tasks with the same structure (ODEs, series, linear systems and limits, 80 runs per adapter, same sandbox), Lamba 0.1 and both seeds were between 2.5% and 6.2% with no significant differences. Neither adapter shipped. A runtime guard that asks the model to stop and answer after a verified value is followed by calls that add no new information rescued 32 of 32 affected runs for seed 1337 and 14 of 14 on 12 held-out integrals, also in the original sandbox; it is not deployed.

The same family in sandbox-B. We re-ran the integral family for the two shipped versions in sandbox-B, 70 runs each (7 tasks × 10). Post-correct rumination is essentially absent for both (Table 5): 0 of 70 for

Table 5: Integral-reconciliation family in sandbox-B (70 runs per model). "Overran then answered": kept calling tools after reaching the value but still gave a final answer.

Model	Post-correct rumination	Collapsed	Correct and stopped	Overran then answered	Budget hit
Lamba 0.1	0/70 (0%)	0	39	31	2
Lamba 0.2	1/70 (1.4%)	1	38[†]	28	1

[†] Plus 3 runs correct only in the final answer.

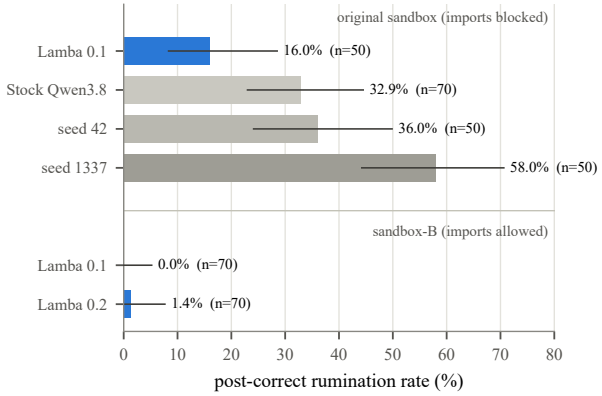


Figure 7: Share of runs on integral-reconciliation tasks that reach the correct value and then exhaust the tool budget without answering, in the original sandbox that blocked imports. In sandbox-B the rate is 0% for Lamba 0.1 and 1.4% for Lamba 0.2 (Table 5).

Lamba 0.1 and 1 of 70 for Lamba 0.2 (Fisher exact $p = 1.0$). The earlier 16% for Lamba 0.1 was therefore a product of the restrictive sandbox, like the tool failures in §9. Lamba 0.2 does not improve on Lamba 0.1 here, and we do not claim that it does. The seed adapters were not re-run in sandbox-B, so their regression is established only in the original sandbox.

10 Comparison with other models

We ran eight other models on the same g150 and eval30 sets (Table 6, Figure 8). External models were called through their providers' APIs with temperature 0 on g150 where the provider accepts it. Our runner sets no reasoning-effort or thinking parameter for them, so each ran at its provider's default reasoning setting, and at provider-default repetition settings. Lamba, by contrast, runs at reasoning effort "medium". Claude models were run through Claude Code in headless mode with the same sandbox exposed as a tool server and extended thinking turned off; part of the Sonnet runs and its g150 run used the API directly, with the provider's default sampling and no thinking. For models that return reasoning in a separate field, we concatenated it with the answer before scoring. Lamba ran at repetition penalty 1.1, which the other models did not use, so the g150 comparison reflects each model as served rather than a matched decoding setup.

On g150, Lamba as served has lower loop rates than the other open-weight models in the table; Claude Sonnet 5.5 has the lowest. On R_{heavy}, the strictest metric, Lamba 0.1 is level with DeepSeek and stock Qwen and lower than GLM ($p = 0.0005$). Because the penalty alone does much of the work (§6.5), we also ran the two same-architecture models at penalty 1.1 (bottom of Table 6). The penalty helps them too, but at matched decoding Lamba 0.2 still has fewer Liquid-style loops than stock Qwen (4 vs 33 of 150, $p = 10^{-6}$) and ThinkingCap (4 vs 17, $p = 0.004$) and fewer budget hits than stock Qwen (53 vs 99, $p = 3 \times 10^{-10}$). Against ThinkingCap at 1.1, the differences in tail repetition (4 vs 11, $p = 0.09$) and budget hits (53 vs 57)

Table 6: g150 loop rates (%, lower is better) and eval30 tool results: correct %, redundant calls after verification (% of runs reaching a verified answer, with that count n_{done}), assistant characters and tool calls per task.

	g150				eval30 (240 runs each)				
Model	L	R_{any}	R_{heavy}	B	Correct	Redund.	n_{done}	Chars	Calls
Lamba 0.2	2.7	2.7	0.0	35.3	98.8	8.8	160	1803	2.00
Lamba 0.1	4.0	2.7	0.0	39.3	98.8	5.5	183	1793	1.92
Qwen3.8-27B (stock)[a]	34.0	22.0	1.3	70.7	96.2	3.9	203	3084	1.34
ThinkingCap-Qwen3.8-27B	18.7	13.3	2.0	49.3	100	5.9	205	1171	1.43
DeepSeek v4.1 Flash	26.0	17.3	0.0	60.0	100	10.1	208	2023	1.78
GLM-5.3 Flash	30.0	26.0	8.0	64.7	91.2	3.2	188	5032	1.35
ILMU v3.1	19.3	18.7	15.3	22.7	75.0	30.4	92	526	4.78
Gemini 3.8 Flash[b]	–	–	–	10.7	100	4.1	197	885	1.18
Claude Sonnet 5.5[c]	0.0	0.0	0.0	4.7	100	0.0	57	540	1.16
Claude Opus 5.5[c]	–	–	–	–	100	1.7	59	722	1.04
Matched decoding: repetition penalty 1.1 for all									
Base model (no adapter)	8.7	4.7	0.0	36.0					
Qwen3.8-27B (stock)	22.0	10.0	0.0	66.0					
ThinkingCap-Qwen3.8-27B	11.3	7.3	0.7	38.0					

[a] A second run of stock Qwen through a different provider gave L 36.0, R_{any} 20.7, R_{heavy} 0.0, B 74.0. [b] Reasoning is not returned, so repetition cannot be scored. [c] Without extended thinking; Opus was not run on g150. GLM, DeepSeek and Gemini are their providers' fast-tier models. Redundancy rates rest on very different n_{done} and are not directly comparable across models.

are not significant. On eval30, accuracy is near the ceiling for most models, and Lamba writes 1.5 to 3.3 times more text per task than Claude, Gemini and ThinkingCap and makes more tool calls. ThinkingCap [1] hits the 4,000-token cap more often than Lamba 0.2 (49% vs 35%), but on prompts both finish its median length is about 740 tokens against 1,661 for Lamba 0.2. Closing this gap likely needs training with a correctness reward and a length cost.

11 Analysis: what the training changes

Training and the penalty overlap, and stack. Table 1 shows the two levers reach similar end points by different routes. The penalty acts on every token at every step, with no change to the model; training changes the model's preferences at loop onsets and leaves decoding untouched. On their own each removes nearly all runaway loops, and their effects add up on the shorter repeats that remain: training on top of the penalty removes about 60% of what the penalty alone leaves. A plausible reading is that the penalty makes repetition expensive everywhere while FTPO makes the model less inclined to start a repeat in the first place; we have not tested this mechanism directly. The practical difference is robustness: a trained model does not depend on users applying the right decoding setting, while the penalty has a narrow useful window (Figure 3: 1.15 already helps less than no penalty, and 1.2 is worse than the base model).

The training data are mostly re-quoting loops, and the effect transfers. Of the 1,298 first-round pairs, 1,038 (80%) come from loops whose repeated window also appears in the prompt: the model restating the question or an option over and over. Our evaluation discards windows that occur in the prompt, so every reduction we report is in text the model wrote itself. The reduction also holds on science and Malay prompts, which share little wording with training prompts: Liquid-style loops fall from 43 to 5 of 85 (Table 4).

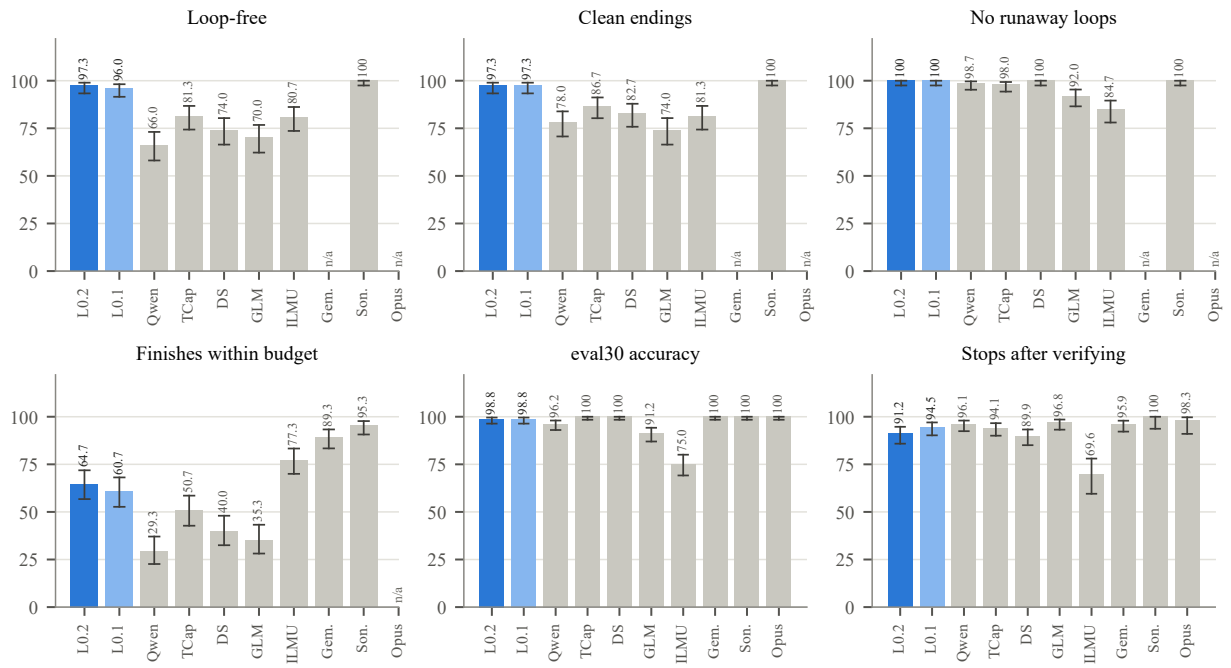


Figure 8: Table 6 as "higher is better" scores (100 minus each rate), with 95% Wilson intervals (150 prompts for g150, 240 runs for eval30 accuracy, n_{done} runs for redundancy; eval30 intervals ignore clustering by task and are therefore too narrow). Lamba 0.2 and 0.1 in blue.

The training position decides the outcome. The first pipeline cut the context one occurrence too early, at the start of the first occurrence. The rejected tokens were then ordinary text (15% were `\n\n`), and training cut the newline rate from 7.11 to 0.03 per 1,000 characters and increased looping. Moving the position to the second occurrence, as Algorithm 1 does, is the difference between the failed and the working adapters.

Loops depend on the whole trajectory. We took looping outputs from one of the adapters of our KL-tie experiments (§12). At the onset of a loop, that adapter and Lamba 0.1 disagree *less* than elsewhere in the same output (7 of 8 cases), and Lamba 0.1 itself loops in 67% of cases when continued from the same prefix, whether it takes over at the onset or up to 100 tokens earlier. A loop is therefore not a single bad token that a local fix can repair: by the time it starts, the context already pulls strongly towards it, consistent with the self-reinforcement described by Xu et al. [16]. Training works by making the model less likely to enter such states, not by correcting them at one point.

Why stronger penalties fail. The penalty lowers the logit of every token already used, which includes the words and symbols a long answer needs. On malay-final, budget hits are 41.3% at penalty 1.0, 36.7% at 1.1, 50.7% at 1.15 and 86.7% at 1.2, and mean output length at 1.2 is more than twice that at 1.0 (17,723 vs 7,879 characters). Beyond 1.1 the model cannot finish; the primary endpoint at 1.2 is worse than the base model's (Figure 3).

Seed robustness. We retrained the last loop stage (v9c) with a different seed, keeping its initialisation and data. Table 7 shows no significant difference on any loop or capability metric. Earlier stages were trained once, so this measures the variance of the final stage only. The two tool-rumination seeds in §9.4 are a

Table 7: Seed replication of the v9c stage (repetition penalty 1.0, g150). Paired exact McNemar p for loop metrics.

	L	R_{any}	R_{heavy}	B	MalayMMLU	A–B gap	GPQA-D
v9c (seed 3407)	22.7	16.7	2.7	41.3	76.50	−12.6	154/198
v9c (seed 2)	18.0	15.3	2.7	42.0	76.51	−13.0	157/198
p	0.23	0.83	1.0	1.0	–	–	–

reminder that seed variance can matter: they behaved very differently on the integral family in the original sandbox.

12 Negative results

- **Tying a shortening run to Lamba 0.1.** SFT on naturally concise correct outputs gave the best capability numbers of any adapter (GPQA-D 162/198) but increased loops. A KL tie to Lamba 0.1 kept the length reduction and lowered the loop cost, but across three seeds the excess loop rate did not reliably stay under the 2-point cap; the effect of the tie strength was smaller than seed-to-seed variation.
- **Length-conditioned repetition penalty.** A decode-time penalty that activates once the output is long and a repeated n-gram is detected was harmful as a replacement for the fixed penalty (R_{any} +29.7 points on science) and neutral to harmful as an addition.
- **Local repair of loops.** Training on Lamba 0.1's continuation at the onset of another adapter's loops was not viable: those continuations also loop in 67% of cases, and starting up to 100 tokens earlier did not help (§11).
- **Structural loops.** Prompts designed to elicit enumeration or rumination loops yielded 2 genuine cases in 486 (0.4%); we stopped.
- **Efficient reasoning.** A 378-pair Malay pilot reduced budget hits by 8 points ($p = 0.023$) without loop regression. Two cross-domain expansions, which chose the stopping point by probing for a correct forced answer, increased R_{any} to 17.3% and 26.0%: on multiple-choice data, forced early answers are often right by chance, so the method taught the model to stop too early. The pilot used outputs from other models as reference lengths and was not shipped.
- **Seed adapters for tool rumination** (§9.4). Not shipped.

13 Findings from live use

The original incident. The prompt that started the investigation was a short question about a Latin phrase. Lamba 0.1 reasoned at length, described itself with a persona and instructions it had not been given, and ended without an answer. Lamba 0.2 answered the same prompt in a few lines. On 60 prompts of the same kind, blind-labelled and then hand-audited, invented identity or instructions appeared in 6 of 60 Lamba 0.1 answers and 3 of 60 Lamba 0.2 answers (Fisher $p = 0.49$). The raw labeller counts were 11 and 3 ($p = 0.043$) before an audit that was designed after seeing them.

Identity. Without a system prompt, Lamba 0.2 does not identify itself as Lamba and in 96% of cases names another model, most often Qwen. With our system prompt it says Lamba 88% of the time. When a client supplies its own system prompt, our server now presents it as application instructions, which helped in a small test (3/4 to 4/4).

Tool follow-through. On 248 held-out prompts at temperature 1.0, Lamba 0.2 calls a tool in 71.8% of cases where one is warranted. It announces a search and then ends its turn without calling the tool in 2.4% of cases, and answers questions about obscure entities without searching in 25.8%.

14 Lessons for practitioners

1. **Get the training position exactly right.** AntiDoom only worked once pairs were taken at the start of the *second* occurrence. One position earlier, the same pipeline taught the model to stop writing newlines.
2. **Mine from the model's own outputs, per domain.** Each round on the adapter's own loops removed loops the previous round left behind, and Malay needed its own round. Where natural loops are rare, model-written prompts with real rollouts supply enough examples.
3. **Run the decoding-only baseline first.** A repetition penalty on the untouched base model turned out to do about as much as our training. Running that baseline at the start, and the full 2×2 of model and decoding, is what tells you what training is actually for.
4. **Select bias checkpoints on large held-out data.** A 664-question validation set showed a stable 2–6 point gap while the full benchmark swung by 25 points across the same checkpoints.
5. **Check the harness before blaming the model.** Most of Lamba 0.1's tool failures, and stock Qwen's, came from a sandbox that blocked imports; so did its 16% rate of post-correct rumination on integrals, which falls to 0% in a working sandbox.
6. **Hand-check and freeze every metric.** Four rounds of real bugs were found by reading fresh outputs, including the detector substitution reported in Table 2.

15 Limitations

- **The penalty does much of the work.** A repetition penalty alone gives a loop reduction comparable to our training (Table 1); the adapters' added value at the serving setting is a further reduction of the remaining short repeats, not of budget hits or the primary endpoint. We did not measure the capability cost of the penalty on the base model.
- **Detector choice.** The pre-registered endpoint misses its relative target on the best-powered test. Our loop detectors are our own, and the Liquid-style detector belongs to the same family as the one used to mine training pairs. The independent checks ran on earlier adapters, and the blind adjudication used a single model as annotator.
- **Selection on test sets.** The repetition penalty was chosen on g150, and the letter-bias checkpoint on the full MalayMMLU.
- **Scale.** One base model, one GPU, 150 to 700 prompts per loop comparison, mostly greedy decoding; one sampled check on 100 prompts agrees.
- **Capability checks.** GPQA-D was run from an ungated mirror with a 4,000-token budget, so 51–59% of answers are cut off. There is no code-domain evaluation, and the tool evaluation is maths only.
- **Lamba 0.2.** No pre-specified non-inferiority margins. The seed adapters' integral regression was measured only in the original sandbox and was not re-run in sandbox-B.
- **Generality.** All results are on one base model that we continued training ourselves; we have not shown the recipe transfers to other bases. Lamba 0.4 uses a different base and a different training

pipeline, so it is not a clean test of transfer, and comparisons between the two reports mix changes in base and in training.

16 How this work was done

Most of the engineering, data generation and labelling was done with two AI systems, while we set the goals and made every ship decision. DeepSeek v4.1 Flash wrote prompt sets, labelled transcripts blind and drafted code. Claude (Opus and Sonnet) planned experiments, checked designs and results, and ran the machine. Errors in proposals from both were caught and corrected along the way; the ones that affected reported numbers are listed in Appendix A.

17 Conclusion

With one GPU and two weeks, a port of AntiDoom mined on the adapter's own outputs and separately per language removed nearly all runaway loops from a 28.8B hybrid Gated DeltaNet model without any change to decoding. A repetition penalty of 1.1 achieves a similar reduction on the untouched base model, and the two together leave the fewest loops of any configuration we tested, fewer than same-architecture models at the same penalty. The pre-registered loop endpoint improved significantly under every configuration, though not by the 30% we targeted. A group-level permutation objective reduced the answer-letter gap substantially, with the caveat that the checkpoint was chosen on the benchmark used to report it. Capability was unchanged within noise. The main open questions are whether the letter result holds on held-out data and how to reduce verbosity, which we expect to need reinforcement learning with a length cost. The identity and tool-use weaknesses found in live use are addressed in Lamba 0.3 and 0.4, which continue this work and are described in a separate report.

Release. Lamba 0.1 and 0.2 are internal research versions and their weights are not released. Lamba 0.4, the version we do release as open weights, is described together with Lamba 0.3 in a separate report, which continues the line of work set out here. What accompanies this report is code: the trainer and its configurations, the mining scripts, the loop and tool evaluation harnesses, and the scoring scripts named in the Reproduction section. We do not release raw model outputs, training data or our internal working notes. The code is at `https://github.com/Vix0007/lamba`, and the Lamba 0.4 weights and model card are at `https://huggingface.co/VixeroAI/Lamba-0.4`.

Author contributions. One researcher at Vixero Labs set the goals, designed and ran all experiments, wrote the evaluation protocols and made every ship decision. AI systems assisted with engineering, data generation, labelling and drafting, as described in the section "How this work was done"; they are not authors.

Broader impact. Fewer runaway loops mean fewer wasted tokens and fewer answers that never arrive, and the Malay work improves a lower-resource language. Lamba still answers questions about obscure entities without checking in about a quarter of cases and does not reliably know its own identity without a system prompt; the model card states both. We do not release benchmark items: GPQA question text is withheld. Lamba is built on a Qwen-family base, as its model card states.

References

[1] BottleCap AI. ThinkingCap-Qwen3.8-27B: the same answers, 37% less thinking. Blog post and model release, `https://bottlecapai.com`, 2026.

[2] Bubblewrap contributors. Bubblewrap: low-level unprivileged sandboxing tool. `https://github.com/containers/bubblewrap`.

[3] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do NOT think that much for 2+3=? on the overthinking of o1-like LLMs. arXiv:2412.21187, 2024.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.14314.

[5] Blessed Guda, Lawrence Francis, Gabrial Zencha Ashungafac, Carlee Joe-Wong, and Moise Busogi. Quantifying and mitigating selection bias in LLMs: A transferable LoRA fine-tuning and efficient majority voting approach. arXiv:2511.21709, 2025.

[6] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. arXiv:1904.09751.

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. arXiv:2106.09685.

[8] Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. CTRL: A conditional transformer language model for controllable generation. arXiv:1909.05858, 2019.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. arXiv:2309.06180.

[10] Sam Paech, Maxime Labonne, Justin Li, Leonie Monigatti, Nathan Ranchin, Tim Seyde, and Sergei Tilga. Reducing doom loops with final token preference optimization. Blog post and code, `https://www.liquid.ai/blog/antidoom`, `https://github.com/Liquid4All/antidoom`, 2026. Liquid AI, 7 July 2026.

[11] Pouya Pezeshkpour and Estevam Hruschka. Large language models sensitivity to the order of options in multiple-choice questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024. arXiv:2308.11483.

[12] Soon Chang Poh, Sze Jue Yang, Jeraelyn Ming Li Tan, Lawrence Leroy Tze Yao Chieng, Jia Xuan Tan, Zhenyu Yu, Foong Chee Mun, and Chee Seng Chan. MalayMMLU: A multitask benchmark for the low-resource Malay language. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

[13] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.18290.

[14] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level Google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

[15] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations*, 2020. arXiv:1908.04319.

[16] Jin Xu, Xiaojiang Liu, Jianhao Yan, Deng Cai, Huayang Li, and Jian Li. Learning to break the loop: Analyzing and mitigating repetitions for neural text generation. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2206.02369.

[17] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. arXiv:2412.06464.

[18] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations*, 2024. arXiv:2309.03882.

[19] Jinquan Zheng, Jia Yuan, Jiacheng Yao, Chenyang Gu, Pujun Zheng, and Guoxiu He. Mitigating selection bias in large language models via permutation-aware GRPO. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. arXiv:2603.21016.

A Corrections

The following numbers appeared in internal notes and were corrected.

- **Primary endpoint detector.** Internal summaries reported the primary endpoint computed with the Liquid-style detector (e.g. 31.2% on 700 prompts) as the pre-registered result. With the pre-registered detector the figure is 22.8% (Table 2).
- **Loop reduction attributed to training.** Earlier drafts credited the full reduction from the base model to the adapter at penalty 1.1. A penalty-only baseline shows the penalty alone achieves a comparable reduction; the adapter's further contribution at the serving setting is limited to the remaining short repeats (§6.5).
- **Integral-family rumination.** The 16% rate for Lamba 0.1 came from a sandbox that blocked imports; in sandbox-B it is 0 of 70 (Table 5).
- **"Lamba 0.2 is 30% less verbose than 0.1."** The 0.2 model folder linked to the base model's generation settings, which lack the repetition penalty, so it was evaluated at 1.0 while 0.1 ran at 1.1. At matched settings they do not differ significantly.
- **"Tool rumination 37.5%."** A metric artifact; see §9. (Unrelated to the 37.5% of runs that never reached the answer in the original sandbox.)
- **Sandbox rescue rate 82.1%.** A number-matching bug omitted one-decimal answers on five tasks. The corrected rate, for a predecessor adapter combined with sandbox-B, is 94.6%.
- **"Stock Qwen shows 0% repetition."** Its reasoning was stored in a separate field the scorer did not read.
- **"0.2's predecessor halves rumination versus 0.1."** True only in the original sandbox.
- **Latin-incident counts.** An early labelling prompt wrongly stated that the system prompt contained nothing about identity. Counts were redone with a corrected rubric.

- **A third guard failure mode.** Empty answers in a synthetic stress test came from artificial tool-call identifiers in the test itself; with real identifiers the failure did not occur.
- **Sonnet 5.5 eval30.** An early partial run (0 of 35 redundant, 501 characters, 1.19 calls) was superseded by the full 240 runs in Table 6.
- **Mean length under a hard cap.** Truncated outputs pull the mean down; length comparisons use paired medians on prompts both models finish.

B Training details

All stages use NF4 QLoRA with the adapter layout of §3, batch size 1 with gradient accumulation, AdamW and a learning-rate warmup over 5–10% of steps. Loop and letter stages use a maximum sequence length of 6,000 tokens; the tool stages use 8,000 and 20,000.

Stage	Initialised from	Training data	Epochs	LR	Eff. batch
v8	base	1,298 FTPO pairs (filtered AntiDoom events)	2	1.5×10^{-5}	16
v9b	v8 (via an intermediate run)	909 own-output rows (hard + soft) + 120 base-tie rows	2	2×10^{-5}	16
v9c	as v9b	v9b data + 1,080 letter-tie rows	2	2×10^{-5}	16
malay-final	v9c	273 Malay rows (weight 0.25) + 600 replay + 200 letter-tie	1	1×10^{-5}	16
Lamba 0.1	malay-final	1,080 rotation groups; step 10 kept	2	1×10^{-5}	16
Seed 1337	Lamba 0.1	about 190 rows: 29 targeted rows (tool rumination, confabulation) + replay ties	2	2×10^{-5}	16
Lamba 0.2	seed 1337	406 DPO pairs + 800 replay rows	4	5×10^{-6}	32

Loss settings: FTPO stages use a logit-clip margin of 1.5 and a logit-matching tether to the base model (weight 2.0 for v8, 0.1 afterwards) with gradient clipping at 5. The letter stage uses hinge 0.4, permutation variance 0.2 and entropy 0.05. Lamba 0.2's loss is given in §9.

C Compute

Everything ran on one NVIDIA L40S (48 GB) with 30 GB of host RAM, with a second L40S for evaluation on a few days. Training stages took between about 15 minutes and a few hours, except the DPO stage of Lamba 0.2, which took 10.5 hours. One g150 evaluation of an adapter takes about an hour, the full MalayMMLU about 20 minutes and GPQA-D about 75 minutes. The work from the first training run to Lamba 0.2, including all failed attempts, took two weeks of wall-clock time on this hardware.

D Reproducibility checklist

- **Do the claims match the results?** Yes. The headline loop claims are stated per metric, and the pre-registered endpoint is reported under the planned and the substituted detector (Table 2).
- **Are limitations discussed?** Yes, in §15, including selection on test sets.
- **Statistical significance.** Paired comparisons use exact McNemar tests; relative reductions carry 95% bootstrap intervals (2,000 resamples over prompts); Figures 2 and 8 show 95% Wilson intervals. With 150 prompts, a rate near 20% has an interval of roughly ± 6 points.

- **Seeds.** The final loop stage was replicated with a second seed (Table 7); other stages were trained once.
- **Pre-registration.** The loop endpoint was fixed before training; one amendment was made before any adapter data were seen, and one detector substitution was made later and is disclosed.
- **Code, data and outputs.** We release the trainer, configurations, mining scripts, evaluation harnesses and scoring scripts. We do not release raw model outputs, training data, or the weights of Lamba 0.1 and 0.2, so the numbers in this report can be re-derived by running the released harness on a model, not recomputed from our outputs. GPQA question text is withheld.
- **Compute.** Appendix C.

E Prompts and harness

g150 loop test. Each prompt is sent as a single user message with no system prompt, through the model's chat template, greedy, with a 4,000-token cap. Prompts are released in `antidoom_data/loop_testset150.jsonl`.

eval30 tool test. Every model, including the external ones, receives the same system prompt (Lamba's serving prompt: an identity line, the current date and the Lamba constitution) and the same two tools, so the harness is identical across models. The tool used in every task is:

```
python_eval: Execute Python code in a sandbox with sympy pre-imported (all its
names available directly, e.g. symbols, Matrix, simplify, expand, solve,
nsimplify). Use this for any exact symbolic math, arithmetic, or verification
- never do hand algebra you can instead compute exactly. No filesystem,
network, or OS access; 15s CPU/wall-clock limit, 1GB memory limit. Use print()
for output.
```

A web-search tool is also offered but is not needed for any task. Two example task prompts:

> *Integral:* "Using python_eval with SymPy, compute the definite integral of 1/(x^2+4) from x=0 to x=2, then verify by numerically integrating the same function over the same bounds (e.g. via a fine Riemann sum or sympy's numerical integration) and confirming the two match."
>
> *ODE:* "Using python_eval with SymPy, solve the ODE y" + y = 0 with y(0)=0, y'(0)=1 exactly and report y(1). Then verify by numerically integrating the ODE (e.g. a hand-written RK4 loop or scipy solve_ivp) from 0 to 1. Report the final value of y(1)."

Runs use temperature 0.8, top-p 0.95 and seed equal to the repetition index, with at most 8 tool rounds followed by one forced answer turn without tools. The Claude Code harness runs at most 9 turns and has no forced answer turn; this never came into play.

GPQA-Diamond. The system message reads: "Reasoning effort is set to xhigh. Please think carefully through the task, validate key assumptions, consider plausible alternatives, and prioritize correctness, consistency, and clarity in the final answer." The user message contains the question and lettered options. At the 4,000-token budget the reasoning is cut and the model is asked for its answer. Question text is not reproduced here.

MalayMMLU. Each item uses the benchmark's prompt (the question and its lettered options, one per line) followed by "Answer:"; the prediction is the option letter with the highest log-probability as the next token.

F Reproduction

- Loop test: `antidoom_data/loop_test_g.py`; loop metrics with `tail_stats_cmp.py`, which refuses to compare files generated at different repetition penalties; the pre-registered endpoint with `loop_compare_v2.py`.
- Tool test: `gen_sandboxb_eval.py` for local models, `gen_ilmu.py` for API models, `gen_claude_sub.py` for Claude Code; scoring with `rumination_metric_v2_2_FROZEN.py`.
- Training configurations: `antidoom_src/configs/`; Lamba 0.2 uses `lamba_ruminconfab_dpo_v2.yaml`.
- Figures in this report: `report/make_figures.py`.